TP 3 : Boids sur grille spatiale

Interactions, affectation, thread pool

Programmation parallèle CPU : C++20, GLFW

Objectifs

— partir d'une simulation de boids déjà clusterisée sur une grille uniforme.

— paralléliser le calcul des forces, là où chaque boid n'écrit que les siennes.

— paralléliser le remplissage des cellules avec un compteur atomique et une mémoire déjà réservée.

— migrer affectation et forces vers un thread pool (mêmes tâches, workers réutilisés).

Prérequis

TP2 (threads, atomiques, pool). Dépendances : GLFW, OpenGL, libepoxy, glm. Le squelette est dans `squelette/tp3`.

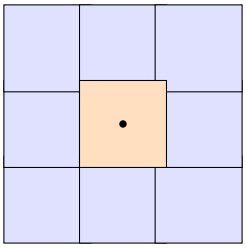
```
cmake -S . -B build && cmake --build build
./build/tp
./build/tp --check
```

La console affiche chaque seconde les moyennes `grille` et `forces`. `--check` compare plusieurs pas à une référence séquentielle, même graine, sans ouvrir de fenêtre. C'est la vérité : toute autre trajectoire est fausse.

1 Exercice 0 : référence (`tp3/00_sequentiel`)

Rien à coder. C'est la mesure de référence.

Des boids évoluent dans une boîte torique. Chaque boid subit trois forces locales : séparation, alignement, cohésion. Calculer ces forces contre tous les autres boids coûte trop cher. On découpe donc le monde en une grille uniforme. Un boid n'interroge que sa cellule et les huit autour. Le rayon d'interaction tient dans ce voisinage : la taille d'une cellule est au moins égale au plus grand rayon.



un boid lit sa cellule et les huit autour

FIGURE 1 – Le rayon d'interaction tient dans le voisinage de la grille.

À chaque pas : on range les boids dans les cellules (`rebuild_grid`), on calcule les forces, on intègre positions et vitesses. Ici, tout est séquentiel. L'affichage passe par GLFW. Notez les temps `grille` et `forces` : ce sont ceux qu'il faudra battre.

2 Exercice 1 : interactions (`tp3/01_interactions`)

La grille est déjà construite, dans un seul fil. Le calcul des forces, lui, peut se partager. Chaque boid écrit seulement `forces[i]`. Deux boids qui écrivent des cases différentes ne se marchent pas dessus. Lire la grille en parallèle ne pose pas de problème : personne ne la modifie pendant ce calcul.

Les boids sont découpés en plages d'indices contiguës, une par worker. Le worker t traite les indices

$$\left[\frac{n\,t}{w},\ \frac{n\,(t+1)}{w}\right).$$

Il appelle `compute_forces_range(begin, end, forces)`. Les forces doivent être finies avant l'intégration : on `join` tous les threads avant de continuer.

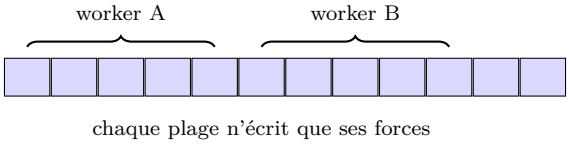


FIGURE 2 – Découpage des boids. La grille, elle, est seulement lue.

Interface du squelette

```
void compute_forces_range(int begin, int end, std::vector<Vec2>& forces) const;
void compute_forces_parallel(std::vector<Vec2>& forces) const;
// worker_count_ = 2 * hardware_concurrency()
```

1. La boucle sur une plage est déjà dans `compute_forces_range`. Elle lit les cellules voisines et écrit `forces[i]` pour $i \in [\mathit{begin}, \mathit{end})$.
2. TODO `compute_forces_parallel`. Découper $[0, n)$ en `worker_count_` plages. Lancer un `std::thread` par plage. Joindre avant de rendre la main.
3. Ne pas protéger `forces` par un mutex : chaque thread écrit une plage disjointe. Un mutex ici ne ferait que ralentir.

Exemple de découpage (même formule que pour une répartition équitable) :

```
const int n = static_cast<int>(boids_.size());
const int w = std::max(1, worker_count_);
for (int t = 0; t < w; ++t) {
  const int begin = n * t / w;
  const int end   = n * (t + 1) / w;
  // workers.emplace_back([&, begin, end] {
  //   compute_forces_range(begin, end, forces);
  // });
}
```

`./build/tp --check` lance le parallèle et la référence séquentielle avec la même graine. Les positions doivent coïncider. Si un thread écrit hors de sa plage, ou si le `join` est oublié, le test échoue.

3 Exercice 2 : affectation (`tp3/02_affectation`)

Ranger les boids dans les cellules se fait aussi en parallèle. Le calcul du numéro de cellule, `cell_index`, ne touche personne. En revanche deux boids peuvent tomber dans la *même* cellule. Si chaque cellule était une `std::vector` et qu'on faisait `push_back` sans protection, deux insertions concurrentes casseraient la liste : même idée que `sum += v` au TP2, trois opérations non atomiques.

Un mutex par cellule corrigerait la course, mais sérialiserait exactement le point qu'on voulait paralléliser. On fait autrement, comme en compute shader : chaque cellule a déjà une tranche fixe dans un grand tableau `cell_ids_`, et un compteur `cell_counts_[c]`.

```
uint prev = atomicAdd(counter, 1u); // GLSL / compute shader
ids[base + prev] = id;
```

`atomicAdd` renvoie l'ancienne valeur du compteur : c'est l'indice du slot libre. En C++20, c'est `std::atomic_ref<int>::fetch_add`. L'incrément est indivisible. Avec `memory_order_relaxed` ça suffit : seul le compteur est partagé pendant l'affectation, et le `join` des workers synchronise avant la lecture des listes.

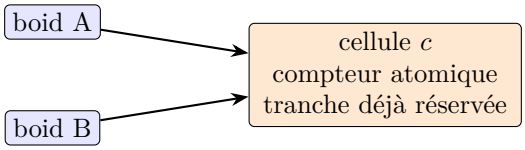


FIGURE 3 – Deux insertions concurrentes. `fetch_add` donne à chacun un `prev` distinct.

Interface du squelette

```
std::vector<int> cell_counts_;       // un compteur par cellule
std::vector<int> cell_ids_;          // ncells * cell_cap_ slots
// cellule c : indices [c * cell_cap_, c * cell_cap_ + cell_counts_[c])
```

1. Au début du pas, remettre `cell_counts_` à zéro.
2. Découper les boids comme à l'exercice 1. Chaque worker calcule `c = cell_index(...)` hors de toute section critique.
3. `TODO rebuild_grid`. Pour chaque boid i de la plage :

```
const int prev =
    std::atomic_ref<int>(cell_counts_[c])
        .fetch_add(1, std::memory_order_relaxed);
if (prev < cell_cap_) {
  cell_ids_[c * cell_cap_ + prev] = i;
}
```

4. Joindre. Ensuite les forces parcourent les slots $0 \le k < \min(\texttt{count}, \texttt{cell_cap_})$ de la cellule c.

Le squelette démarre avec un `cell_counts_[c]++` non atomique : c'est volontairement incorrect. `--check` exige que chaque boid apparaisse exactement une fois, et que le pas parallèle reste celui de la référence séquentielle. Comparez aussi le temps `grille` à l'exercice 1 : l'affectation ne doit plus être négligeable seulement parce qu'elle est protégée, elle doit être courte.

4 Exercice 3 : thread pool (`tp3/03_thread_pool`)

Créer w `std::thread` à chaque pas, pour $w = 2\times$ le nombre de cœurs, a un coût. Les workers font le même travail qu'aux exercices 1 et 2 ; seule la façon de les lancer change. On réutilise un pool déjà démarré.

Le pool fourni n'est pas une usine à `packaged_task` et `future`. C'est une file circulaire :

— `write_` : tête d'écriture (producteur : le thread principal) ;

— `read_` : tête de lecture (les workers se la disputent en `fetch_add`) ;

— `work_` : sémaphore POSIX, un `sem_post` par tâche poussée ;

— `done_` : sémaphore qui réveille `wait()` quand plus aucune tâche n'est en cours (`pending_` repasse à 0).

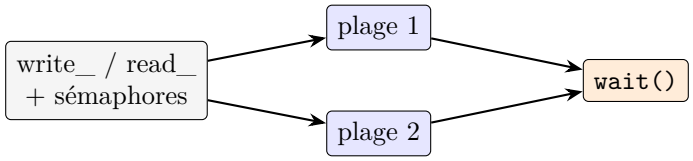


FIGURE 4 – Soumettre toutes les plages d'abord. `wait()` ensuite.

On ne change pas l'algorithme : même découpage, même `fetch_add` pour l'affectation. On remplace `std::thread` + `join` par `pool_.submit` puis un seul `pool_.wait()`.

Attention

`wait()` doit *dormir* (sémaphore), pas tourner en `yield`. Sinon le thread principal vole des cœurs aux workers, et les forces paraissent plus lentes alors que le travail est identique.

Interface du squelette

```
class ThreadPool {
 public:
  void submit(std::function<void()> job);
  void wait();  // bloque tant que pending_ > 0
};
```

1. Le squelette démarre comme l'exercice 2 : atomiques + `std::thread`. Ça compile et ça passe `--check`.
2. `TODO rebuild_grid`. Pour chaque plage, `pool_.submit([...])` avec le même `fetch_add`. Après la boucle : `pool_.wait()`.
3. `TODO compute_forces_parallel`. Même migration : `submit` des plages, puis `wait()`.
4. Ne pas appeler `wait()` juste après chaque `submit` : sinon il n'y a plus de chevauchement.

Exemple :

```
for (int t = 0; t < w; ++t) {
  const int begin = n * t / w;
  const int end  = n * (t + 1) / w;
  pool_.submit([this, begin, end, &forces] {
    compute_forces_range(begin, end, forces);
  });
}
pool_.wait();
```

Comparez les temps `grille` / `forces` avec l'exercice 2. Sur la grille (petites tâches fréquentes), le pool doit gagner ou coller. Sur les forces (peu de gros jobs), les temps doivent être du même ordre : le voisinage n'a pas changé, seule la façon de lancer les workers a changé. `--check` doit encore passer.

5 Livrables

Les quatre étapes compilent. `--check` passe pour les étapes 1, 2 et 3. Les temps console permettent de comparer séquentiel, forces parallèles, affectation atomique, puis pool.